

Министерство образования Республики Беларусь  
Учреждение образования  
«Белорусский государственный университет информатики  
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра программного обеспечения информационных технологий

Учебная дисциплина «Метрология, стандартизация и сертификация в  
информационных технологиях»

Отчет

по практической работе №1  
«Метрики размера программ»

Подготовили:

Сулимов А.А. гр. 410902  
Козинцев М.Л. гр. 410902

Проверил:

Болтак С.В.

Минск 2026

**Цель:** изучить метрики размера программ на примере метрики Холстеда.

**Задание:** написать программу-парсер. Парсер может быть реализован на любом языке программирования и должен иметь графический интерфейс. Для анализа взять код на заданном языке. На основе анализа программа должна рассчитать метрики Холстеда согласно примера 1 в материале выше: 6 базовых и 3 расширенных. Результат работы программы - таблица, как в примере, в которой должны отображаться 6 основных метрик Холстеда. Отдельно выводятся 3 расширенные метрики.

Код анализируемой программы на языке Scala:

```
object DataAnalyzer {

  def calcMean(arr: Array[Double], len: Int): Double = {
    var sum: Double = 0.0
    var i: Int = 0
    while (i < len) {
      sum = sum + arr(i)
      i = i + 1
    }
    return sum / len
  }

  def findMax(arr: Array[Double], len: Int): Double = {
    var maxVal: Double = arr(0)
    var i: Int = 1
    while (i < len) {
      if (arr(i) > maxVal) {
        maxVal = arr(i)
      }
      i = i + 1
    }
    return maxVal
  }

  def getFactorial(n: Int): Int = {
    var res: Int = 1
    var k: Int = 1
    while (k <= n) {
```

```
    res = res * k
    k = k + 1
  }
  return res
}
```

```
def bubbleSort(arr: Array[Double], len: Int): Unit = {
  var i: Int = 0
  var j: Int = 0
  var tmp: Double = 0.0
  while (i < len) {
    j = 0
    while (j < len - i - 1) {
      if (arr(j) > arr(j + 1)) {
        tmp = arr(j)
        arr(j) = arr(j + 1)
        arr(j + 1) = tmp
      }
      j = j + 1
    }
    i = i + 1
  }
}
```

```
def showInfo(m: Double, mx: Double, f: Int): Unit = {
  println(m)
  println(mx)
  println(f)
}
```

```
def main(args: Array[String]): Unit = {
  var size: Int = 5
  var data: Array[Double] = new Array[Double](size)

  data(0) = 4.5
  data(1) = 2.1
  data(2) = 7.8
  data(3) = 1.2
}
```

```
data(4) = 5.6
```

```
var variable: String = "Hello"
```

```
bubbleSort(data, size)
```

```
var meanRes: Double = calcMean(data, size)
```

```
var maxRes: Double = findMax(data, size)
```

```
var factNum: Int = 4
```

```
var factRes: Int = getFactorial(factNum)
```

```
var threshold: Double = 5.0
```

```
if (maxRes > threshold) {
```

```
    showInfo(meanRes, maxRes, factRes)
```

```
} else {
```

```
    println(factRes)
```

```
}
```

```
}
```

```
}
```

Результаты расчета базовых метрик Холстеда вручную представлены в таблице 1.

| j  | Оператор | f1j | i  | Операнд | f2i |
|----|----------|-----|----|---------|-----|
| 1  | =        | 40  | 1  | i       | 16  |
| 2  | :        | 34  | 2  | 1       | 13  |
| 3  | var      | 17  | 3  | arr     | 13  |
| 4  | { }      | 16  | 4  | j       | 11  |
| 5  | Double   | 15  | 5  | data    | 9   |
| 6  | Int      | 15  | 6  | len     | 8   |
| 7  | ,        | 10  | 7  | 0       | 6   |
| 8  | +        | 9   | 8  | k       | 5   |
| 9  | [ ]      | 6   | 9  | size    | 5   |
| 10 | def      | 6   | 10 | sum     | 4   |
| 11 | Array    | 6   | 11 | maxVal  | 4   |
| 12 | while    | 5   | 12 | res     | 4   |
| 13 | <        | 4   | 13 | tmp     | 3   |

|    |              |   |    |              |   |
|----|--------------|---|----|--------------|---|
| 14 | println      | 4 | 14 | maxRes       | 3 |
| 15 | >            | 3 | 15 | factRes      | 3 |
| 16 | return       | 3 | 16 | 0.0          | 2 |
| 17 | if           | 3 | 17 | n            | 2 |
| 18 | Unit         | 3 | 18 | m            | 2 |
| 19 | -            | 2 | 19 | mx           | 2 |
| 20 | calcMean     | 2 | 20 | f            | 2 |
| 21 | findMax      | 2 | 21 | 4            | 2 |
| 22 | getFactorial | 2 | 22 | meanRes      | 2 |
| 23 | bubbleSort   | 2 | 23 | factNum      | 2 |
| 24 | showInfo     | 2 | 24 | threshold    | 2 |
| 25 | String       | 2 | 25 | "Hello"      | 1 |
| 26 | <=           | 1 | 26 | DataAnalyzer | 1 |
| 27 | *            | 1 | 27 | args         | 1 |
| 28 | /            | 1 | 28 | variable     | 1 |
| 29 | object       | 1 | 29 | 5            | 1 |
| 30 | main         | 1 | 30 | 4.5          | 1 |
| 31 | new          | 1 | 31 | 2.1          | 1 |
|    |              |   | 32 | 2            | 1 |
|    |              |   | 33 | 7.8          | 1 |
|    |              |   | 34 | 3            | 1 |
|    |              |   | 35 | 1.2          | 1 |
|    |              |   | 36 | 5.6          | 1 |
|    |              |   | 37 | 5.0          | 1 |

$$\eta_1 = 31$$

$$N_1 = 219$$

$$\eta_2 = 37$$

$$N_2 = 138$$

$$\text{Словарь программы: } \eta = 31 + 37 = 68$$

$$\text{Длина программы: } N = 219 + 138 = 357$$

$$\text{Объем программы: } V = 357 * \log_2 68 = 2173.22 \text{ бит}$$

Программа-парсер написана на языке Python с использованием библиотеки tkinter для создания графического интерфейса.

Результаты расчета метрик Холстеда с помощью парсера представлены на рисунках 1-2.

| j       | Оператор     | f1j      | i       | Операнд      | f2i      |
|---------|--------------|----------|---------|--------------|----------|
| 1       | =            | 40       | 1       | i            | 16       |
| 2       | :            | 34       | 2       | arr          | 13       |
| 3       | var          | 17       | 3       | 1            | 13       |
| 4       | { }          | 16       | 4       | j            | 11       |
| 5       | Double       | 15       | 5       | data         | 9        |
| 6       | Int          | 15       | 6       | len          | 8        |
| 7       | ,            | 10       | 7       | 0            | 6        |
| 8       | +            | 9        | 8       | k            | 5        |
| 9       | def          | 6        | 9       | size         | 5        |
| 10      | Array        | 6        | 10      | sum          | 4        |
| 11      | [ ]          | 6        | 11      | maxVal       | 4        |
| 12      | while        | 5        | 12      | res          | 4        |
| 13      | <            | 4        | 13      | tmp          | 3        |
| 14      | println      | 4        | 14      | maxRes       | 3        |
| 15      | return       | 3        | 15      | factRes      | 3        |
| 16      | if           | 3        | 16      | 0.0          | 2        |
| 17      | >            | 3        | 17      | n            | 2        |
| 18      | Unit         | 3        | 18      | m            | 2        |
| 19      | calcMean     | 2        | 19      | mx           | 2        |
| 20      | findMax      | 2        | 20      | f            | 2        |
| 21      | getFactorial | 2        | 21      | 4            | 2        |
| 22      | bubbleSort   | 2        | 22      | meanRes      | 2        |
| 23      | -            | 2        | 23      | factNum      | 2        |
| 24      | showInfo     | 2        | 24      | threshold    | 2        |
| 25      | String       | 2        | 25      | DataAnalyzer | 1        |
| 26      | object       | 1        | 26      | args         | 1        |
| 27      | /            | 1        | 27      | 5            | 1        |
| 28      | <=           | 1        | 28      | 4.5          | 1        |
| 29      | *            | 1        | 29      | 2.1          | 1        |
| 30      | main         | 1        | 30      | 2            | 1        |
| 31      | new          | 1        | 31      | 7.8          | 1        |
|         |              |          | 32      | 3            | 1        |
|         |              |          | 33      | 1.2          | 1        |
|         |              |          | 34      | 5.6          | 1        |
|         |              |          | 35      | variable     | 1        |
|         |              |          | 36      | "Hello"      | 1        |
|         |              |          | 37      | 5.0          | 1        |
| ---     | ---          | ---      | ---     | ---          | ---      |
| η1 = 31 |              | N1 = 219 | η2 = 37 |              | N2 = 138 |

Рисунок 1 – Результат работы парсера

Словарь программы ( $\eta = \eta_1 + \eta_2$ ): 68  
Длина программы ( $N = N_1 + N_2$ ): 357  
Объем программы ( $V = N * \log_2(\eta)$ ): 2173.22 бит

Рисунок 2 – Расширенные метрики Холстеда